

```

        perror("ioctl");

    close(fd);
    exit(0);
}
r &= ~(1<<3); // reset D3 pin
r |= (inp << 3); //set D3 if inp is 1

if(ioctl(fd, PPWDATA, &r) !=0) // write to the port again
    perror("ioctl");

```

You can connect your devices to all the data port pins and control them. The ID of bit 0 is D0, bit 1 is D1, bit 7 is D7, etc. At the end, remember to release and close the port:

```

if(ioctl(fd,PPRELEASE) !=0)
    perror("ioctl");

close(fd);

```



Note: This user space code has to be executed with root permissions.

Using a parallel port from a device driver

Now, let's look at how to control devices connected to a parallel port from a device driver. If you have worked with *char* device and *misc* device, you will know that the latter is the easiest way to create a character device. For those who are new to device drivers, let me give you a few details. When you register a *char* driver, you will need to have a major number and minor number that you have to combine using *MKDEV* to get a *dev_t* value. Then you will need to register this *dev_t* with *register_chrdev_region()* and after that, you will need to call *cdev_init()* and *cdev_add()*. Finally, you have to create the device node in the */dev* folder, manually. We can avoid all this hard work if we use *misc_device*. We just need to call *misc_register()* and all the tasks are done automatically, including the device node creation in the */dev* folder. But for that we need to define a *structure_miscdevice*.

Let's look at the basic parts of our driver with just the header files, *init* and *exit*.

```

#include<linux/kernel.h>
#include<linux/module.h>
#include<linux/miscdevice.h>
#include<linux/device.h>
#include<linux/fs.h>
#include<linux/slab.h>
#include <linux/uaccess.h>
#include <linux/parport.h>

int __init init_m(void)
{

```

```

    return parport_register_driver(&mydevice_driver);
}

void __exit cleanup_m(void)
{
    parport_unregister_driver(&mydevice_driver);
}

module_init(init_m);
module_exit(cleanup_m);
MODULE_LICENSE("GPL");

```

In the *init* function, we are just registering our driver with the parallel port. But we have not yet defined our driver.

```

static struct parport_driver mydevice_driver = {
    .name = "mydevice",
    .attach = mydevice_attach,
    .detach = mydevice_detach,
};

```

mydevice_attach and *mydevice_detach* are the entry and exit points for our driver. Whenever Linux detects any parallel port, it will call the drivers and attach a function where we register our device with the parallel port, and try to claim it. The remaining part of the code will work only if we claim it.

Assuming that we have claimed the parallel port successfully, let's register *miscdevices*:

```

static void mydevice_attach(struct parport *port)
{
    int i,err;

    if (port->number != 0) // we connect to lpt1
        return;
    if (pprt) {
        pr_err("port->number=%d already registered!\n",port-
>number);
        return;
    }

    pprt = parport_register_device(port, "mydevice", NULL,
NULL, NULL, 0, (void *)&pprt);
    if (pprt == NULL) {
        pr_err("port->number=%d, parport_register_device()
failed\n", port->number);
        return;
    }

    if (parport_claim(pprt)) {
        pr_err("could not claim access to parport.
Aborting.\n");
        goto err_unreg_device;
    }
}

```